CMSC 22300: Functional Programming

Final Project, Part 1: Proposal

Allison Lee

Probabilistic Vigenère Cryptanalysis with Monads and N-Gram Language Models

Project Overview

For my final project, I propose to extend Programming Assignments (PA) 2 and 3 (Vigenère cipher assignments) into a probabilistic, multi-candidate decryption system that incorporates scoring mechanisms. The previous Vigenère assignments produced a deterministic best-guess key and plaintext using unigram frequency analysis. The limitation of this implementation is that for ciphertexts that are more ambiguous or complex, it can result in incorrect results and offer very little insight into the reasoning process. My project will aim to move beyond a deterministic decryption model to a probabilistic model that generates, ranks, and outlines the logic behind multiple candidates. Through this shift, I hope to make the Vigenère cryptanalysis process into a more flexible and interpretable system that highlights probabilistic and principled reasoning under uncertainty.

Various Difficulty Categories

- Easy Category: Structured Refactoring and Multi-Candidate Ranking

  The Easy category takes the Vigenère cipher assignments and architecturally restructures it to incorporate monads (specifically, State and Writer monads). State monads will be used to keep track of and update analysis information, such as tracking candidate key positions, maintaining frequency counts, and sequentially accumulating probabilities while scoring the decrypted plaintext (which also then scores the corresponding keys). Writer monads will be used to log scoring decisions, record intermediate probability contributions, and provide explanations for ranking decisions. Incorporating State and Writer monads makes the analysis process more maintainable and transparent, which successfully sets a clean foundation for the probabilistic extensions in the Medium and Challenge categories. The Easy implementation will generate multiple candidate keys, produce multiple candidate decrypted plaintext outputs, and then score and rank candidates using the same unigram language model used in PA 2 and 3. Therefore, instead of outputting a single plaintext guess, the program will return the top 3 ranked decryptions with associated scores.

- Medium Category: Probabilistic Vigenère and Likelihood Modeling

  The Medium category extends the Easy category by incorporating explicit probabilistic modeling. Instead of selecting a single key candidate for every key position, the Medium implementation will assign probabilities to possible shifts at each key position, and then combine these probabilities across the key positions, computing the likelihoods for the full candidate decryptions. The outputs would then be ranked using both key likelihood and language likelihood. The Medium category implementation is a jump from the Easy category, which uses deterministic multi-candidate output and scores by numeric sorting, because it explicitly

represents the uncertainty using probability distributions and compositional probability calculations—possibly through a custom probability type, inspired by PA 5 (Probability Monad).

- Challenge Category: N-Gram Language Models

  The Challenge category extends the Medium category (probabilistic Vigenère) by replacing the unigram language model with an n-gram language model. Unlike the unigram frequency analysis approach, n-gram models use conditional probability to estimate: $P(letter \mid previous\ n\ -\ 1\ letters)$. This would significantly improve the ranking of plaintext because it better encapsulates the contextual structure of English, instead of treating letters independently. Therefore, this implementation will compute log-likelihood using n-gram probabilities and return ranked candidate decryptions.

Additional Topics and Resources

To complete this project, I may need to explore the following topics:

- N-gram language modeling → I have learned some basics of Natural Language Processing (NLP) in other courses, but I will need to learn and research about N-gram language models in my own time. Some useful online resources I have found are:
  https://www.geeksforgeeks.org/nlp/n-gram-in-nlp/
  https://medium.com/@abhishekjainindore24/n-grams-in-nlp-a7c05c1aff12
  https://web.stanford.edu/~jurafsky/slp3/3.pdf
  I will continue using more resources to individually learn about n-gram language modeling and NLP as the project progresses.
- Efficient lookup structures (such as dictionaries or map-based frequency tables) → I will refer to our in-class lectures, specifically Lecture 8: Containers, as well as the Haskell documentation on Data.Map.
- Log-probability computations → I have already learned log-probability computations through other courses, but I have also found this useful resource by Chris Piech from Stanford University: https://chrispiech.github.io/probabilityForComputerScientists/en/part1/log_probabilities/
- State monad → Lectures 13 and 14
- Writer monad → Lecture 16
- Probability monad → PA 5 and in-class lectures

I believe that most of the topics and knowledge needed for completion of my project will be learned through lectures and programming assignments as the course progresses, so my primary independent study will focus on n-gram language modeling, if I am advised to pursue the Challenge category of my project. Ultimately, given the scope and timing constraints of the project, I believe that with the combination of class lectures, office hours, and online resources, learning the concepts is reasonable.